

Aprendizaje Automático (Machine Learning)

Técnicas de Inteligencia Artificial

Prof. Flavio Prieto
email: faprieto@unal.edu.co

INGENIERÍA MECATRONICA
Facultad de Ingeniería
Universidad Nacional de Colombia Sede Bogotá



17 de marzo de 2026

Introducción.

- ▶ El aprendizaje automático (Machine Learning, ML) es una rama de la inteligencia artificial que permite a los sistemas aprender a partir de datos, sin ser programados explícitamente para cada tarea.
- ▶ Su objetivo principal es **identificar patrones** y **hacer predicciones o decisiones automáticas** basadas en datos históricos.
- ▶ El ML se puede dividir en diferentes paradigmas, según la forma en que el modelo aprende.

Tipos de aprendizaje en ML

Existen tres grandes tipos de aprendizaje:

- ▶ **Aprendizaje supervisado**
- ▶ **Aprendizaje no supervisado**
- ▶ **Aprendizaje por refuerzo**

Cada uno se aplica según el tipo de datos disponibles y el objetivo del modelo.

Aprendizaje supervisado.

El modelo aprende a partir de un conjunto de datos que incluye las entradas (X) y las salidas deseadas (y).

Ejemplos:

- ▶ Clasificación: identificar si un correo es spam o no.
- ▶ Regresión: predecir el precio de una vivienda.

Su objetivo es aprender una función que relacione X con y .

Aprendizaje no supervisado.

El modelo trabaja solo con datos de entrada (X), sin etiquetas conocidas.

Ejemplos:

- ▶ Agrupamiento (clustering): segmentar clientes según su comportamiento.
- ▶ Reducción de dimensionalidad: simplificar los datos manteniendo la información esencial.

Su objetivo es descubrir patrones o estructuras ocultas en los datos.

Aprendizaje por refuerzo.

Un agente interactúa con un entorno, tomando decisiones para maximizar una recompensa acumulada.

Ejemplos:

- ▶ Juegos: como AlphaGo.
- ▶ Robótica: controlar el movimiento de un brazo robótico.

El aprendizaje se basa en prueba y error, ajustando la estrategia según las recompensas obtenidas.

Aprendizaje Supervisado

- ▶ Es un tipo de aprendizaje automático donde el modelo aprende a partir de datos etiquetados.
- ▶ Busca aprender una función

$$f : X \rightarrow Y$$

a partir de un conjunto de entrenamiento $\{(x_i, y_i)\}_{i=1}^N$.

- ▶ Dos grandes tareas:
 - ▶ **Clasificación:** cuando y es una categoría.
 - ▶ **Regresión:** cuando y es un valor numérico continuo.

Componentes del Aprendizaje Supervisado:

- ▶ **Conjunto de datos:** entradas $x_i \in \mathbb{R}^n$, salidas $y_i \in \mathbb{R}$ o etiquetas.
- ▶ **Modelo:** función parametrizada que se ajusta con los datos.
- ▶ **Función de pérdida:** mide el error entre la predicción y el valor real.
- ▶ **Algoritmo de entrenamiento:** optimiza los parámetros del modelo minimizando la pérdida.

Técnicas de Clasificación Supervisada.

- ▶ Regresión logística
- ▶ Máquinas de Vectores de Soporte (SVM)
- ▶ K-Nearest Neighbors (KNN)
- ▶ Árboles de decisión
- ▶ Random Forest
- ▶ Gradient Boosting Machines (GBM)
- ▶ Naive Bayes

Técnicas de Regresión Supervisada.

- ▶ Regresión lineal
- ▶ Support Vector Regression (SVR)
- ▶ KNN Regressor
- ▶ Árboles de regresión
- ▶ Random Forest Regressor
- ▶ Gradient Boosting Regressor

Técnicas de **Clasificación** Supervisada.

Regresión Logística

- ▶ Modelo de aprendizaje supervisado para **clasificación binaria**, donde se estima la probabilidad de que una entrada pertenezca a una clase.
- ▶ Utiliza una combinación lineal de variables transformada por la función sigmoide.

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad \text{donde } z = \mathbf{w}^T \mathbf{x}$$

Ejemplo: clasificar si un correo es spam o no spam.

Fundamento matemático.

Función de Decisión:

- ▶ La regresión logística modela la probabilidad de que la variable dependiente $y \in \{0, 1\}$ ocurra dado un vector de características $\mathbf{x}$:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$

donde:

- ▶ $\sigma(z) = \frac{1}{1 + e^{-z}}$ es la función sigmoide.
 - ▶ $\mathbf{w} \in \mathbb{R}^d$ son los pesos del modelo.
 - ▶ b es el sesgo o intercepto.
-
- ▶ Esta función transforma cualquier valor real en un valor entre 0 y 1, lo cual **permite interpretarlo como una probabilidad.**

Función de Pérdida (Entropía Cruzada):

- ▶ Para ajustar los parámetros del modelo se minimiza la siguiente función de pérdida sobre N muestras:

$$\mathcal{L}(\mathbf{w}, b) = - \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

donde $\hat{y}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b)$ es la probabilidad predicha para la muestra i .

Interpretación:

- ▶ Esta función **penaliza fuertemente las predicciones erróneas con alta confianza** y promueve probabilidades bien calibradas.

Optimización - Derivadas Parciales:

- ▶ Las derivadas de la función de pérdida respecto a los parámetros se usan para actualizar los pesos:

$$\frac{\partial \mathcal{L}}{\partial w_j} = \sum_{i=1}^N (\hat{y}_i - y_i) x_{ij}, \quad \frac{\partial \mathcal{L}}{\partial b} = \sum_{i=1}^N (\hat{y}_i - y_i)$$

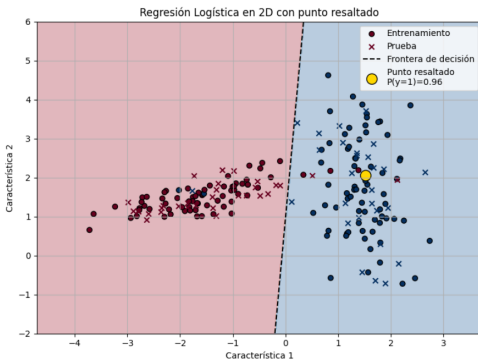
Actualización.

- ▶ A través del gradiente descendente:

$$w_j \leftarrow w_j - \eta \frac{\partial \mathcal{L}}{\partial w_j}, \quad b \leftarrow b - \eta \frac{\partial \mathcal{L}}{\partial b}$$

donde η es la tasa de aprendizaje.

Ejemplo.



Dado el modelo $\hat{y} = 0.143 + 2.215x_1 - 0.151x_2$, para $x = (1.52034231 \quad 2.06774802)^\top \Rightarrow \hat{y} \approx 0.961$.

Interpretación: muy alta probabilidad de que pertenezca a la clase positiva.

Ver código python.

Máquinas de Vectores de Soporte (SVM)

- ▶ Modelo de aprendizaje supervisado para **clasificación binaria** que busca encontrar el hiperplano que **maximiza el margen** entre dos clases.
- ▶ La decisión se basa en una combinación lineal de las características:

$$\hat{y} = \text{sign}(\mathbf{w}^T \mathbf{x} + b)$$

Ejemplo: clasificar si una imagen representa un gato o un perro.

Fundamento matemático.

Función de Decisión:

- ▶ Dado un conjunto de entrenamiento $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$, con $y^{(i)} \in \{-1, +1\}$:

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$$

- ▶ El objetivo es encontrar el hiperplano $\mathbf{w}^\top \mathbf{x} + b = 0$ que **maximiza el margen**:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{sujeeto a} \quad y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1$$

- ▶ En su formulación original (lineal separable), **solo se utilizan los vectores de soporte** que se ubican sobre los márgenes.
- ▶ El margen se define como la distancia entre la frontera

$$\boxed{\mathbf{w}^\top \mathbf{x} + b = 0} \text{ y los márgenes } \boxed{\mathbf{w}^\top \mathbf{x} + b = \pm 1}.$$

Optimización (dual):

- ▶ Se utiliza programación cuadrática para resolver el problema dual:

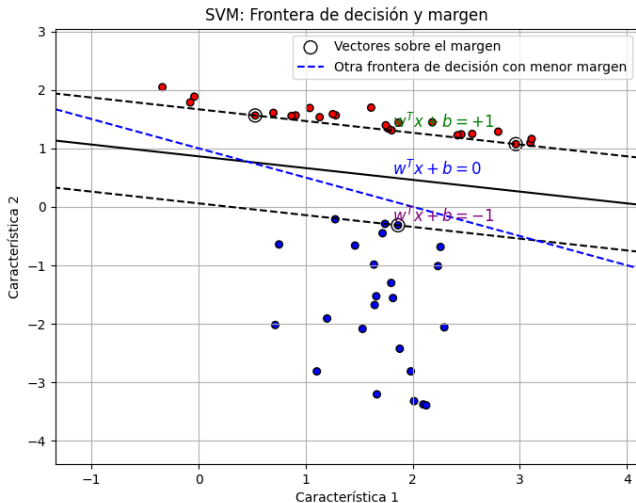
$$\max_{\alpha} \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i,j=1}^N \alpha_i \alpha_j y^{(i)} y^{(j)} \langle \mathbf{x}^{(i)}, \mathbf{x}^{(j)} \rangle$$

- ▶ Sujeto a:

$$0 \leq \alpha_i \leq C, \quad \sum_{i=1}^N \alpha_i y^{(i)} = 0$$

- ▶ Los vectores con $\alpha_i > 0$ son los **vectores de soporte**.

Frontera de decisión y margen.



Función de Pérdida (SVM con Margen Suave).

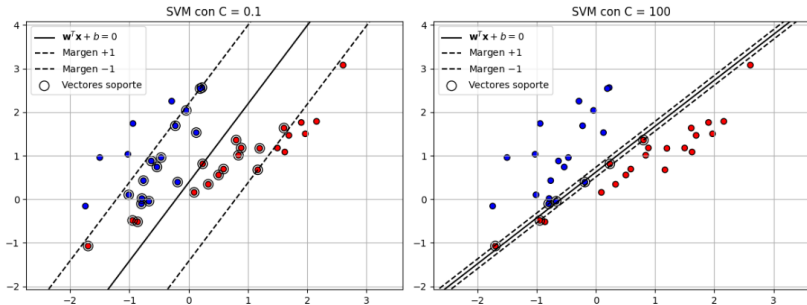
- ▶ Para permitir errores (ej. datos no separables), se introduce una penalización:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N \xi_i \quad \text{sueto a} \quad y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1 - \xi_i$$

- ▶ donde $\xi_i \geq 0$ es la variable de holgura, y
- ▶ C controla el balance entre maximizar el margen y penalizar errores (tolerancia al sobreajuste).
- ▶ Si $\xi_i = 0$: el punto está correctamente clasificado y fuera del margen.
- ▶ Si $0 < \xi_i < 1$: el punto está dentro del margen, pero bien clasificado.
- ▶ Si $\xi_i > 1$: el punto está mal clasificado.

Efecto del hiperparámetro C .

Comparación del efecto de C en SVM



- ▶ Cuando C es grande: **se penaliza fuertemente el error**, el modelo busca un **margen estrecho** que clasifique correctamente todos los datos.
- ▶ Cuando C es pequeño: **se permite violar el margen**, priorizando un **margen amplio** aunque algunos puntos se clasifiquen incorrectamente.

Extensión a Problemas No Lineales.

- ▶ Las SVM lineales solo pueden separar clases cuando los datos son linealmente separables.
- ▶ En muchos casos reales, esto no es posible.

Solución: Transformación a Espacios de Mayor Dimensión.

- ▶ La idea clave es transformar los datos de entrada a un espacio de mayor dimensión **donde una separación lineal sea posible**:

$$\phi : \mathbb{R}^n \rightarrow \mathbb{R}^m, \quad m \gg n$$

- ▶ Esta transformación se realiza implícitamente utilizando funciones núcleo (“kernels”).

Producto Escalar en Espacios Transformados.

- ▶ En lugar de calcular explícitamente $\phi(\mathbf{x})$, se utiliza una función núcleo K que calcula el producto escalar en el espacio transformado:

$$K(\mathbf{x}, \mathbf{x}') = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle$$

Ejemplos Comunes de Núcleos:

- ▶ Núcleo Polinomial: $K(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^\top \mathbf{x}' + c)^d$
- ▶ Núcleo Gaussiano (RBF): $K(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\sigma^2}\right)$
- ▶ Núcleo Sigmoidal (o Tangente Hiperbólica):
 $K(\mathbf{x}, \mathbf{x}') = \tanh(\kappa \mathbf{x}^\top \mathbf{x}' + \theta)$

Optimización en SVM con Kernels.

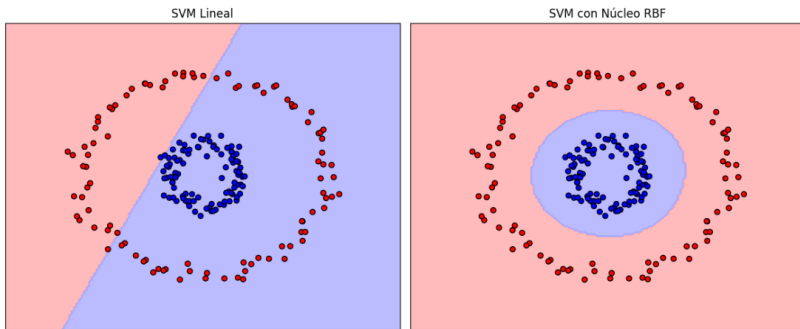
Se resuelve el **problema dual**:

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

sueto a: $0 \leq \alpha_i \leq C, \quad \sum_{i=1}^n \alpha_i y_i = 0$

- ▶ Las variables α_i determinan los **vectores de soporte**.
- ▶ El sesgo b se calcula con los vectores de soporte.
- ▶ El **producto escalar** se reemplaza por una función **kernel** $K(x_i, x_j)$, que representa un espacio de mayor dimensión.
- ▶ La **frontera de decisión** es no lineal en el espacio original, pero lineal en el espacio transformado.

Ejemplo.



- ▶ A la izquierda: datos no separables linealmente.
- ▶ A la derecha: separación lograda en el espacio transformado con un núcleo RBF.

Ver código python.

K-Nearest Neighbors (KNN)

- ▶ Modelo de aprendizaje supervisado basado en instancias para **clasificación** y **regresión**.
- ▶ Para un nuevo punto, la predicción se basa en la mayoría de clases entre sus k vecinos más cercanos.
- ▶ **No construye explícitamente un modelo durante el entrenamiento.**

Ejemplo: clasificar si una imagen representa un gato o un perro según imágenes similares.

Fundamento matemático.

- ▶ Dado un conjunto de entrenamiento $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$, donde $y^{(i)} \in \{1, 2, \dots, C\}$.
- ▶ Para una nueva muestra $\mathbf{x}$, se calcula su distancia a todos los puntos del conjunto de entrenamiento.
- ▶ Se identifican los k vecinos más cercanos:

$\mathcal{N}_k(\mathbf{x}) =$ conjunto de k muestras más cercanas a $\mathbf{x}$

- ▶ La predicción se realiza por mayoría de votos:

$$\hat{y} = \text{moda}(\{y^{(j)} : \mathbf{x}^{(j)} \in \mathcal{N}_k(\mathbf{x})\})$$

KNN y la Regla de Bayes

- ▶ La **regla de Bayes** define el clasificador óptimo en términos de minimizar el error esperado:

$$\hat{y} = \arg \max_{c \in \mathcal{C}} P(y = c \mid \mathbf{x})$$

- ▶ Es decir, **se asigna a cada punto la clase con mayor probabilidad posterior.**
- ▶ **KNN aproxima esta probabilidad** mediante la frecuencia relativa de clases entre los k vecinos más cercanos:

$$P(y = c \mid \mathbf{x}) \approx \frac{1}{k} \sum_{j \in \mathcal{N}_k(\mathbf{x})} \mathbf{1}(y^{(j)} = c)$$

- ▶ Donde $\mathcal{N}_k(\mathbf{x})$ es el conjunto de los k vecinos más cercanos a $\mathbf{x}$.

Clasificación basada en la Regla de Bayes

La **Regla de Bayes** proporciona una forma de calcular la probabilidad de que un patrón pertenezca a una clase, dado un conjunto de datos:

$$P(C_k | \mathbf{x}) = \frac{P(\mathbf{x} | C_k)P(C_k)}{P(\mathbf{x})}$$

- ▶ $P(C_k | \mathbf{x})$: Probabilidad posterior de la clase C_k dado el punto $\mathbf{x}$.
- ▶ $P(\mathbf{x} | C_k)$: Verosimilitud del punto $\mathbf{x}$ bajo la clase C_k .
- ▶ $P(C_k)$: Probabilidad a priori de la clase C_k .
- ▶ $P(\mathbf{x})$: Evidencia total de los datos.

Esta es la base de clasificadores como Naive Bayes y KNN.

¿Cómo KNN aplica la Regla de Bayes?

- ▶ KNN estima $P(C_k | \mathbf{x})$ mediante una frecuencia relativa:

$$P(C_k | \mathbf{x}) \approx \frac{\text{Número de vecinos de clase } C_k}{K}$$

- ▶ Se seleccionan los K vecinos más cercanos a $\mathbf{x}$.
 - ▶ Se cuenta cuántos de esos K pertenecen a cada clase C_k .
 - ▶ Se predice la clase con mayor probabilidad posterior estimada.
-
- ▶ **No necesita estimar explícitamente:** $P(\mathbf{x} | C_k)$ ni $P(\mathbf{x})$, sino que los aproxima directamente desde los datos.

Ventajas del Enfoque Bayesiano en KNN.

- ▶ KNN implementa una forma **no paramétrica** de la regla de Bayes.
- ▶ Su precisión depende directamente del número K y la distribución local de los datos.
- ▶ No requiere suposiciones sobre la forma funcional de las distribuciones.
- ▶ Estima las probabilidades condicionales de forma empírica.

Conclusión

KNN no modela explícitamente la verosimilitud ni la probabilidad a priori, pero las incorpora implícitamente al aplicar la regla de Bayes basada en frecuencia relativa local.

Distancias más comunes.

- ▶ **Euclidiana:**

$$d(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_{i=1}^n (x_i - x'_i)^2}$$

- ▶ **Manhattan:**

$$d(\mathbf{x}, \mathbf{x}') = \sum_{i=1}^n |x_i - x'_i|$$

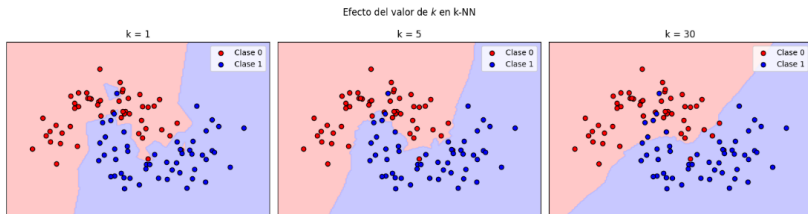
- ▶ **Minkowski (generalización):**

$$d(\mathbf{x}, \mathbf{x}') = \left(\sum_{i=1}^n |x_i - x'_i|^p \right)^{1/p}$$

- ▶ Otras: distancia de Mahalanobis, distancia coseno, Hamming, etc.

Elección del parámetro k .

- ▶ El número de vecinos k es un hiperparámetro fundamental.
- ▶ **Valores pequeños** de $k \rightarrow$ modelo más flexible, más sensible al ruido (sobreajuste).
- ▶ **Valores grandes** de $k \rightarrow$ modelo más estable, pero puede suavizar demasiado las fronteras (subajuste).



Ventajas y desventajas de KNN.

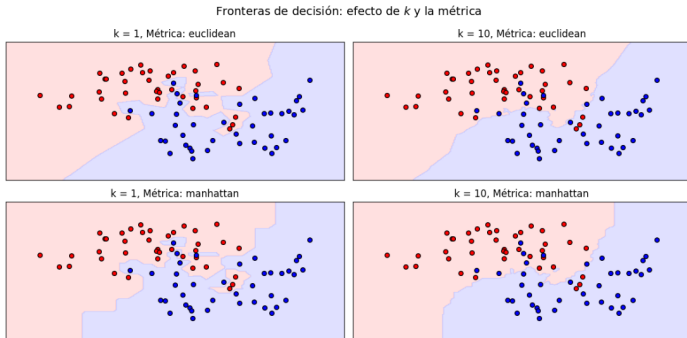
Ventajas:

- ▶ Simple y fácil de implementar.
- ▶ No requiere entrenamiento explícito.
- ▶ Adaptable a problemas multiclase.

Desventajas:

- ▶ **Costoso en tiempo de predicción** (revisa todos los puntos de entrenamiento).
- ▶ Sensible a la escala de los datos y ruido.
- ▶ No maneja bien datos con muchas dimensiones (“maldición de la dimensionalidad”).

Ejemplo gráfico: frontera de decisión.



- ▶ La forma de la frontera depende fuertemente del valor de k y de la métrica utilizada.
- ▶ Fronteras complejas para k pequeño; fronteras suaves para k grande.

Ver código python.

Árboles de Decisión

- ▶ Modelo de aprendizaje supervisado usado para **clasificación y regresión**.
- ▶ Representa un proceso de decisiones en forma de **estructura jerárquica** (nodos y ramas).
- ▶ Cada nodo interno corresponde a una prueba sobre un atributo, cada rama a un resultado de la prueba, y cada hoja a una predicción.

Ejemplo: clasificar tipos de flores iris en función del largo y ancho de los pétalos.

Fundamento Matemático.

- ▶ Dado un conjunto de entrenamiento $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$, el árbol se construye **dividiendo recursivamente los datos en subconjuntos homogéneos**.
- ▶ La función de decisión se define como:

$$\hat{y} = f(\mathbf{x}), \quad f : \mathbb{R}^n \rightarrow \mathcal{Y}$$

donde f está determinada por las reglas del árbol.

- ▶ El objetivo es encontrar divisiones que maximicen la **pureza** de los nodos.
- ▶ El proceso de partición se detiene cuando se cumple un criterio de parada (profundidad máxima, número mínimo de muestras, o pureza máxima).

Medidas de Impureza.

- ▶ Para medir la calidad de una partición, se usan métricas de impureza.

Índice de Gini:

$$G(t) = 1 - \sum_{k=1}^K p_{k|t}^2$$

Entropía:

$$H(t) = - \sum_{k=1}^K p_{k|t} \log_2 p_{k|t}$$

donde

- ▶ $p_{k|t}$ es la proporción de muestras de la clase k en el nodo t .
- ▶ Un nodo es puro si todas sus muestras pertenecen a una sola clase.

Criterio de División.

- ▶ La reducción de impureza se calcula como:

$$\Delta I = I(\text{padre}) - \sum_j \frac{N_j}{N} I(\text{hijo}_j)$$

- ▶ $I(\cdot)$: medida de impureza (Gini o Entropía).
 - ▶ N : número de muestras en el nodo padre.
 - ▶ N_j : número de muestras en el nodo hijo j .
-
- ▶ Se elige la división que **maximiza** ΔI .

Ejemplo de un Árbol de Decisión.

- ▶ Cada nodo representa una pregunta de división (ej. “Estudió horas extra”).
- ▶ Las hojas representan la predicción final.

Conjunto de datos:

Predicción de si un estudiante aprueba (Sí/No) según si estudió horas extra.

Horas extra	Aprobó
Sí	Sí
Sí	Sí
Sí	No
No	No
No	No

Cálculo de entropía del nodo raíz:

$$H = - \left(\frac{2}{5} \log_2 \frac{2}{5} + \frac{3}{5} \log_2 \frac{3}{5} \right) \approx 0.97$$

División: atributo “Horas extra”.

- ▶ Nodo hijo (Sí): clases [2 Sí, 1 No], entropía

$$H_{\text{Sí}} = - \left(\frac{2}{3} \log_2 \frac{2}{3} + \frac{1}{3} \log_2 \frac{1}{3} \right) \approx 0.92$$

- ▶ Nodo hijo (No): clases [0 Sí, 2 No], entropía

$$H_{\text{No}} = 0$$

Ganancia de información:

$$\Delta H = 0.97 - \left(\frac{3}{5} \cdot 0.92 + \frac{2}{5} \cdot 0 \right) \approx 0.42$$

Conclusión: la mejor división inicial es “Horas extra”.

Cálculo de Gini del nodo raíz:

$$G = 1 - \left(\left(\frac{2}{5} \right)^2 + \left(\frac{3}{5} \right)^2 \right) = 1 - \left(\frac{4}{25} + \frac{9}{25} \right) = 1 - \frac{13}{25} = \frac{12}{25} \approx 0.48$$

División: atributo “Horas extra”.

- ▶ Nodo hijo (Sí): clases [2 Sí, 1 No], índice de Gini

$$G_{\text{Sí}} = 1 - \left(\left(\frac{2}{3} \right)^2 + \left(\frac{1}{3} \right)^2 \right) = 1 - \left(\frac{4}{9} + \frac{1}{9} \right) = 1 - \frac{5}{9} = \frac{4}{9} \approx 0.44$$

- ▶ Nodo hijo (No): clases [0 Sí, 2 No], índice de Gini

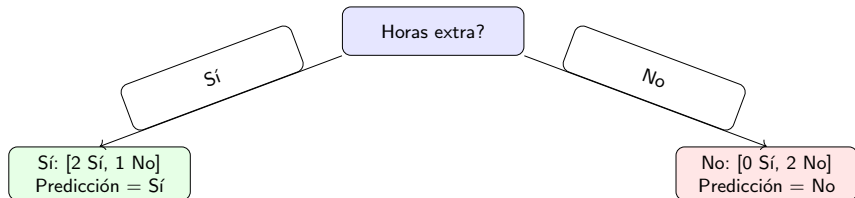
$$G_{\text{No}} = 1 - (0^2 + 1^2) = 0$$

Reducción de impureza (ganancia en Gini):

$$\Delta G = 0.48 - \left(\frac{3}{5} \cdot 0.44 + \frac{2}{5} \cdot 0 \right) = 0.48 - 0.266 \dots \approx 0.21$$

Conclusión: la mejor división inicial también es “Horas extra” según el índice de Gini.

Ejemplo Gráfico del Árbol (un nivel).



- ▶ La raíz divide los datos según el atributo “Horas extra”.
- ▶ Nodo izquierdo: mayoría clase Sí.
- ▶ Nodo derecho: nodo puro, clase No.

Sobreajuste y Poda.

- ▶ Un árbol crecido sin restricciones tiende a sobreajustar.
- ▶ Para controlar esto, se usan:
 - ▶ **Criterios de parada:**
 - ▶ limitar la profundidad máxima,
 - ▶ número mínimo de muestras por nodo, o
 - ▶ pureza mínima.
 - ▶ **Poda:** eliminar ramas poco significativas para mejorar la generalización.
- ▶ La complejidad del árbol puede regularse mediante un parámetro de penalización α :

$$C(T) = \text{Error}(T) + \alpha |T|$$

donde $|T|$ es el número de hojas.

Ventajas y Desventajas.

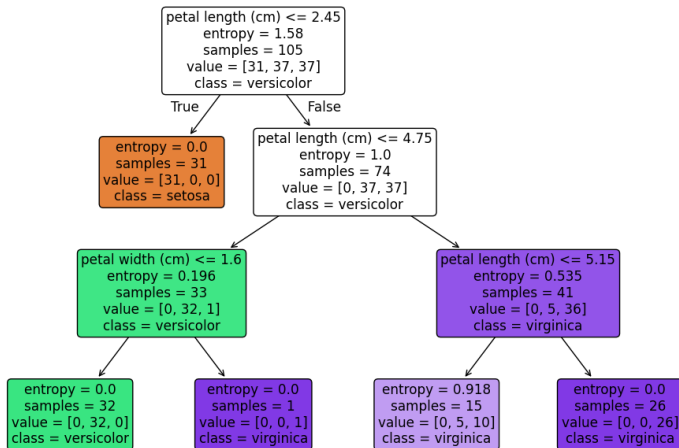
Ventajas

- ▶ Fácil de interpretar y visualizar.
- ▶ Capaz de manejar datos numéricos y categóricos.
- ▶ No requiere normalización de características.

Desventajas

- ▶ Sensibles a pequeñas variaciones en los datos (alta varianza).
- ▶ Pueden generar árboles muy complejos que sobreajustan.
- ▶ Menor capacidad predictiva frente a modelos ensamblados (Random Forest, Gradient Boosting).

Ejemplo gráfico.



Ver código python.

Random Forest (Bosques Aleatorios)

- ▶ Modelo de **aprendizaje supervisado** basado en un conjunto de árboles de decisión.
- ▶ Utiliza la técnica de **ensamble** para mejorar la precisión y reducir el sobreajuste.
- ▶ Cada árbol se entrena con una muestra **bootstrap** de los datos y con una selección aleatoria de atributos.

Ejemplo: clasificar especies de flores a partir de medidas de pétalos y sépalos.

Fundamento Matemático.

- ▶ Dado un conjunto de entrenamiento $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$, se construyen B árboles de decisión:

$$\hat{y}_b = f_b(\mathbf{x}), \quad b = 1, 2, \dots, B$$

- ▶ La predicción final del Random Forest se obtiene por **votación mayoritaria** (clasificación) o **promedio** (regresión):

$$\hat{y} = \text{moda}\{\hat{y}_1, \hat{y}_2, \dots, \hat{y}_B\}$$

Construcción del Bosque.

- ▶ Cada árbol se entrena de manera diferente gracias a dos fuentes de aleatoriedad:
 1. **Bootstrap**: se selecciona una muestra aleatoria (con reemplazo) del conjunto de datos.
 2. **Selección aleatoria de atributos**: en cada nodo, la división se busca entre un subconjunto aleatorio de características.
- ▶ Este procedimiento reduce la correlación entre los árboles y mejora la generalización.

Importancia de las Variables.

- ▶ Random Forest permite estimar la **importancia de cada variable**:
 - ▶ **Promedio de la reducción de impureza** (Gini o entropía) producida por esa variable a lo largo de todos los árboles.
 - ▶ O mediante la **reducción en la precisión del modelo** al permutar aleatoriamente los valores de la variable.
- ▶ Esto ayuda a interpretar el modelo y seleccionar características relevantes.

Ejemplo Ilustrativo.

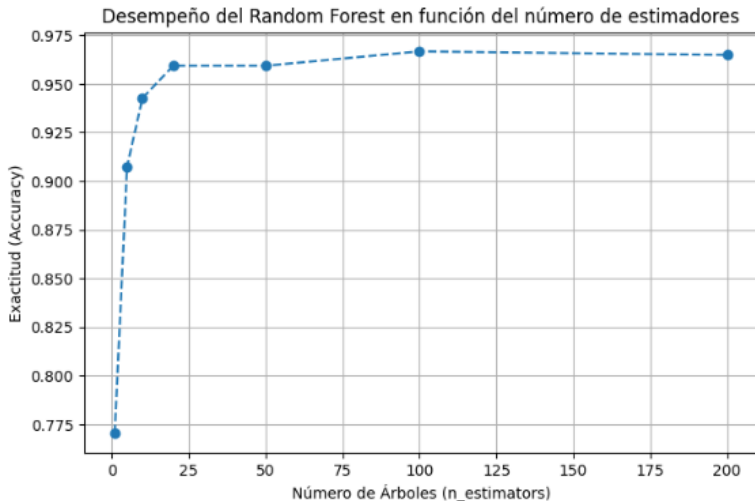
- ▶ Supóngase un conjunto de datos binario (Sí/No).
- ▶ Se entrenan múltiples árboles de decisión con diferentes muestras y subconjuntos de atributos.
- ▶ Cada árbol emite una predicción.
- ▶ El Random Forest combina todas las predicciones y toma la clase con más votos.

Predicción final: la mayoría de los árboles predice “Sí”, entonces el Random Forest clasifica como “Sí”.

Sobreajuste y Generalización.

- ▶ Los árboles individuales son propensos al sobreajuste, pero al combinar muchos de ellos:
 - ▶ Se reduce la **varianza**.
 - ▶ Se mejora la **precisión de predicción**.
- ▶ El error se estabiliza conforme aumenta el número de árboles.
- ▶ No obstante, el modelo pierde interpretabilidad comparado con un único árbol.

Desempeño del clasificador Random Forest en función del número de estimadores.



Ventajas y Desventajas.

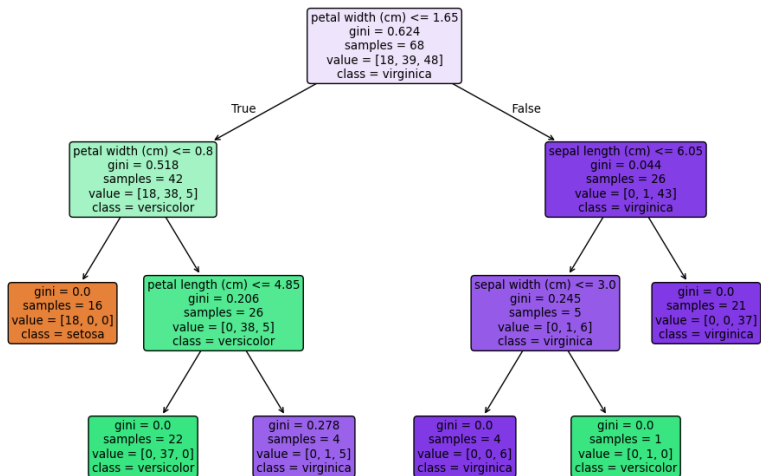
Ventajas

- ▶ Alta precisión en clasificación.
- ▶ Robusto frente a ruido y sobreajuste.
- ▶ Puede manejar grandes cantidades de datos y muchas variables.
- ▶ **Estima la importancia de las variables.**

Desventajas

- ▶ Menor interpretabilidad frente a un único árbol.
- ▶ Entrenamiento y predicción más costosos computacionalmente.
- ▶ No siempre adecuado para problemas en tiempo real.

Ejemplo gráfico: Uno de los árboles del Random Forest



Random Forest e Importancia de Características

- ▶ La técnica **Random Forest** permite no solo realizar clasificación y regresión, sino también **analizar la importancia de las características**.
- ▶ Cada árbol del bosque se construye con subconjuntos aleatorios de datos y variables.
- ▶ El criterio de división de un nodo está basado en una medida de **impureza**, como la entropía $H(t)$ o el índice de Gini $G(t)$:
 - ▶ $H(t) = - \sum_{k=1}^K p_{k|t} \log_2 p_{k|t}$
 - ▶ $G(t) = 1 - \sum_{k=1}^K p_{k|t}^2$
donde p_k es la proporción de elementos de clase k en el nodo t .
- ▶ A partir de estas reducciones de impureza, el modelo determina cuáles variables son más relevantes.

Métodos para Medir la Importancia de la Variable.

1. Disminución Media de Impureza (MDI):

- ▶ Cada vez que un nodo divide los datos con una característica, la impureza (Gini o Entropía) se reduce.
- ▶ Esa reducción se asigna a la característica usada en la división.
- ▶ El proceso se repite en todos los nodos de todos los árboles y luego se **suman las contribuciones**.
- ▶ Finalmente, las importancias se **normalizan** para que su suma sea igual a 1.

$$Imp(f_j) = \frac{1}{T} \sum_{t=1}^T \sum_{n \in N_{j,t}} p(n) \Delta i(n)$$

donde:

- ▶ T = número de árboles,
- ▶ $N_{j,t}$ = nodos en el árbol t que usan la característica f_j ,
- ▶ $p(n)$ = proporción de muestras que llegan al nodo n ,
- ▶ $\Delta i(n)$ = reducción de impureza en el nodo n .

Cálculo de $\Delta i(n)$:

- ▶ En un árbol de decisión (y en Random Forest), una división en un nodo n separa los datos en dos o más subconjuntos según una característica.
- ▶ Medidas de Impureza:

$$Gini(n) = 1 - \sum_{k=1}^K p_k^2; \quad H(n) = - \sum_{k=1}^K p_k \log_2(p_k)$$

donde p_k es la proporción de la clase k en el nodo n .

- ▶ **Reducción de Impureza en el nodo n :**

$$\Delta i(n) = i(n) - \left(\frac{N_{left}}{N_n} i(left) + \frac{N_{right}}{N_n} i(right) \right)$$

donde:

- ▶ $i(n)$ = impureza antes de la división,
- ▶ $i(left), i(right)$ = impurezas de los nodos hijo,
- ▶ N_n = número de muestras en n ,
- ▶ N_{left}, N_{right} = muestras en cada hijo.

2. Disminución Media de Precisión (MDA):

- ▶ También llamada **Permutation Importance**.
- ▶ La idea es medir cuánto empeora el desempeño del modelo cuando se rompe la relación entre una característica y la variable objetivo.
- ▶ Para una característica f_j :
 - ▶ Se calcula la precisión del modelo con los datos originales: $Acc_{original}$.
 - ▶ Se permutan aleatoriamente los valores de f_j en el conjunto de prueba.
 - ▶ Se evalúa de nuevo el modelo con los valores permutados, obteniendo $Acc_{perm(j)}$.
- ▶ La importancia se define como:

$$Imp(f_j) = \frac{1}{R} \sum_{r=1}^R \left(Acc_{original}^{(r)} - Acc_{perm(j)}^{(r)} \right)$$

donde R es el número de repeticiones de la permutación para reducir la varianza.

- ▶ Una mayor caída en la precisión implica que la característica f_j es más relevante.

▶ Ventajas:

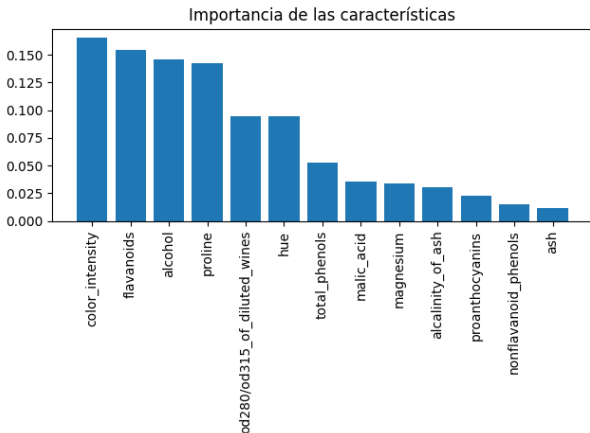
- ▶ Permite identificar las variables más influyentes en el modelo.
- ▶ Útil en problemas con un gran número de características.
- ▶ Facilita procesos de **selección de variables**, reduciendo dimensionalidad.

▶ Limitaciones:

- ▶ La medida MDI puede estar sesgada hacia variables con muchas categorías o mayor variabilidad.
- ▶ La medida MDA es más robusta, aunque computacionalmente más costosa.
- ▶ En presencia de variables altamente correlacionadas, la importancia puede dispersarse entre ellas.

Ejemplo de la importancia de las características

- ▶ Base de datos **wine** compuesta por 13 características.
- ▶ Técnica: Disminución Media de Impureza (MDI).



Ver código python.

Gradient Boosting Machines (GBM)

- ▶ Técnica de **ensamble** basada en árboles de decisión.
- ▶ Construye el modelo de manera **aditiva y secuencial**.
- ▶ Cada nuevo árbol corrige los **errores** de los árboles anteriores.
- ▶ Especialmente efectivo para problemas de **clasificación y regresión**.

Ejemplo: Clasificar correos electrónicos como *spam/no spam*.

Clasificador Débil

- ▶ Modelo sencillo con capacidad predictiva limitada.
- ▶ En GBM suelen ser **árboles poco profundos** (stumps).
- ▶ Individualmente son poco precisos, pero en conjunto forman un modelo fuerte.

Características

- ▶ Árboles con 1 a 5 niveles.
- ▶ Capturan solo parte de la estructura de los datos.
- ▶ Precisión apenas superior al azar.
- ▶ Se construyen de forma secuencial, corrigiendo errores previos.

Ejemplo: un stump divide el espacio en dos regiones; al combinar muchos stumps $\Rightarrow$ modelo potente.

Fundamento Matemático.

- ▶ Dados datos $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$, el modelo se construye de forma aditiva:

$$F_M(\mathbf{x}) = \sum_{m=1}^M \gamma_m h_m(\mathbf{x}),$$

donde h_m son árboles de decisión débiles y γ_m sus pesos.

- ▶ El modelo se ajusta minimizando una **función de pérdida** mediante **gradiente descendente**:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) - \eta \nabla L(y, F_{m-1}(\mathbf{x})),$$

con η = tasa de aprendizaje.

Función de pérdida en clasificación.

- ▶ En clasificación binaria se utiliza comúnmente la **pérdida logística**:

$$L(y, F(\mathbf{x})) = \log(1 + e^{-yF(\mathbf{x})}),$$

donde $y \in \{-1, +1\}$.

- ▶ El gradiente negativo corresponde a los **residuos pseudo-residuales**:

$$r^{(i)} = -\frac{\partial L(y^{(i)}, F(\mathbf{x}^{(i)}))}{\partial F(\mathbf{x}^{(i)})}.$$

Proceso iterativo del GBM.

1. Inicializar el modelo con una predicción constante $F_0(\mathbf{x})$.
2. Para cada iteración $m = 1, \dots, M$:
 - ▶ Calcular los pseudo-residuales $r^{(i)}$.
 - ▶ Ajustar un árbol débil $h_m(\mathbf{x})$ a los $r^{(i)}$.
 - ▶ Calcular el peso óptimo γ_m .
 - ▶ Actualizar el modelo:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta\gamma_m h_m(\mathbf{x}).$$

Regularización y sobreajuste.

- ▶ GBM puede sobreajustar si no se controla.
- ▶ Estrategias comunes:
 - ▶ Reducir la **tasa de aprendizaje** η .
 - ▶ Limitar la **profundidad de los árboles**.
 - ▶ Usar **submuestreo** de datos y variables (stochastic GBM).
 - ▶ Ajustar el número de iteraciones M .

Ventajas y Desventajas.

Ventajas

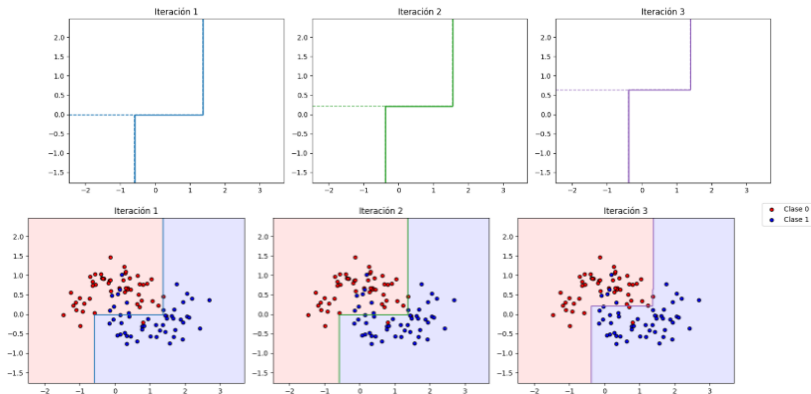
- ▶ Gran capacidad predictiva.
- ▶ Maneja datos heterogéneos (numéricos y categóricos).
- ▶ Control del sesgo y varianza mediante regularización.

Desventajas

- ▶ Mayor complejidad computacional que un solo árbol.
- ▶ Sensible a hiperparámetros (requiere ajuste cuidadoso).
- ▶ Menos interpretable que un árbol individual.

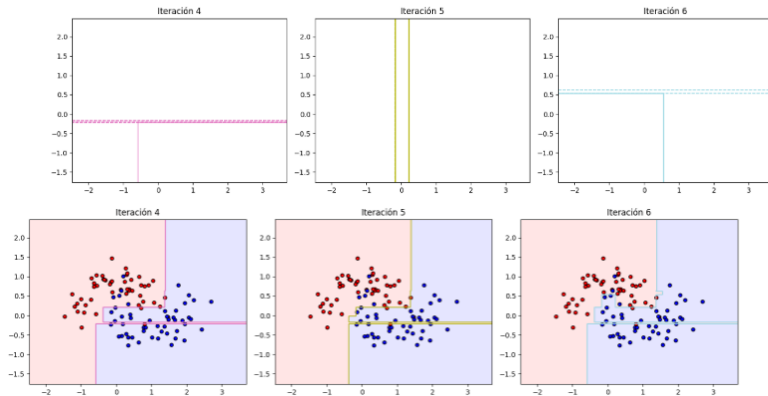
Ejemplo gráfico del GBM.

- ▶ Cada árbol intenta corregir los errores de clasificación de los anteriores.
- ▶ El resultado es una frontera de decisión más precisa y robusta.



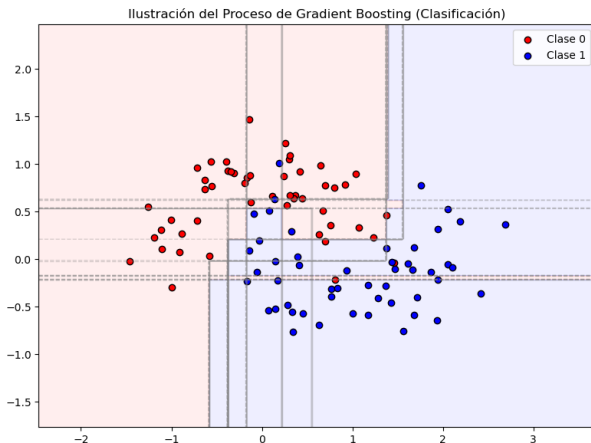
Los primeros árboles se enfocan en casos fáciles.

- ▶ Los siguientes corrigen los errores en los puntos mal clasificados.



Ver código python.

► Clasificador completo:



Ver código python.

Naive Bayes

- ▶ Es un clasificador probabilístico basado en el **Teorema de Bayes**.
- ▶ Se asume que las características son **condicionalmente independientes** dado la clase.
- ▶ Es muy eficiente y funciona bien incluso con pocos datos.

Ejemplo: clasificar correos como “spam” o “no spam” usando palabras como características.

Teorema de Bayes.

- ▶ Para una clase C_k y características $\mathbf{x} = (x_1, x_2, \dots, x_d)$:

$$P(C_k | \mathbf{x}) = \frac{P(\mathbf{x} | C_k) P(C_k)}{P(\mathbf{x})}$$

- ▶ Donde:
 - ▶ $P(C_k)$: probabilidad a priori de la clase.
 - ▶ $P(\mathbf{x} | C_k)$: probabilidad condicional de los atributos dado la clase.
 - ▶ $P(C_k | \mathbf{x})$: probabilidad posterior.
- ▶ La predicción se hace con:

$$\hat{y} = \arg \max_k P(C_k | \mathbf{x})$$

Suposición de Independencia Condicional.

- ▶ La independencia condicional simplifica el cálculo de $P(\mathbf{x} \mid C_k)$:

$$P(\mathbf{x} \mid C_k) = \prod_{j=1}^d P(x_j \mid C_k)$$

- ▶ Esto hace que el modelo sea muy **eficiente computacionalmente**, incluso con muchas características.
- ▶ La suposición “naive” (ingenua) no siempre es cierta, pero el clasificador sigue funcionando sorprendentemente bien en muchos casos prácticos.

Tipos de Naive Bayes

- ▶ **GaussianNB:** asume que los atributos continuos siguen una distribución normal.
- ▶ **MultinomialNB:** se usa para conteos discretos (por ejemplo, frecuencia de palabras en texto).
- ▶ **BernoulliNB:** se usa para atributos binarios (presencia o ausencia de una característica).

Gaussian Naive Bayes

- Supone que cada característica x_j sigue una distribución normal para cada clase C_k :

$$P(x_j | C_k) = \frac{1}{\sqrt{2\pi\sigma_{kj}^2}} \exp\left(-\frac{(x_j - \mu_{kj})^2}{2\sigma_{kj}^2}\right)$$

- La probabilidad condicional conjunta se obtiene por independencia:

$$P(\mathbf{x} | C_k) = \prod_{j=1}^d P(x_j | C_k)$$

- Predicción final:

$$\hat{y} = \arg \max_k P(C_k) \prod_{j=1}^d P(x_j | C_k)$$

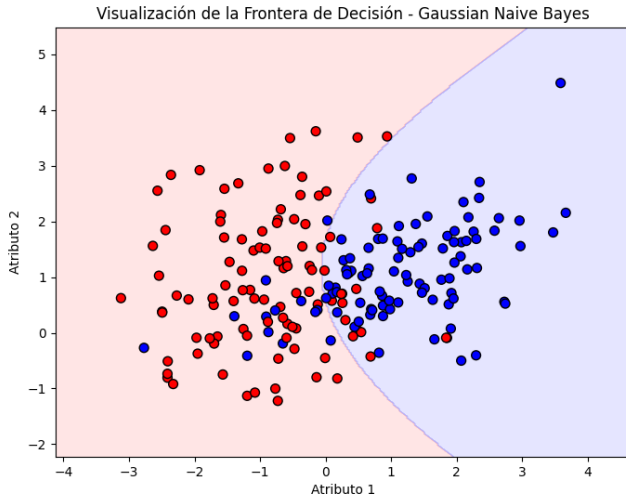
Ejemplo: Gaussian Naive Bayes.

- ▶ Supóngase un conjunto de datos con dos clases y dos características continuas.
- ▶ Cada clase C_k tiene una distribución normal:

$$P(x_j | C_k) = \frac{1}{\sqrt{2\pi\sigma_{kj}^2}} \exp\left(-\frac{(x_j - \mu_{kj})^2}{2\sigma_{kj}^2}\right)$$

- ▶ La predicción se hace evaluando la probabilidad posterior y seleccionando la clase con mayor valor.

Visualización de la Frontera de Decisión.



► Las regiones coloreadas muestran la clase predicha.

Multinomial Naive Bayes

- ▶ Se usa para atributos discretos, típicamente conteos:

$$P(\mathbf{x} \mid C_k) = \frac{(\sum_j x_j)!}{\prod_j x_j!} \prod_{j=1}^d \theta_{kj}^{x_j}$$

- ▶ Donde:

- ▶ x_j es el conteo de la característica j .
- ▶ $\theta_{kj} = P(x_j \mid C_k)$ es la probabilidad de la característica j en la clase C_k .

- ▶ Predicción final:

$$\hat{y} = \arg \max_k P(C_k) \prod_{j=1}^d \theta_{kj}^{x_j}$$

Multinomial Naive Bayes: Clasificación de Textos

- ▶ Técnica probabilística basada en conteos de palabras.
- ▶ Supone independencia condicional entre palabras dado la clase.
- ▶ Predice la clase $\hat{y}$ que maximiza:

$$\hat{y} = \arg \max_k P(C_k) \prod_{j=1}^d P(x_j | C_k)$$

Paso 1: Conjunto de Entrenamiento y Clases

- ▶ Conjunto de entrenamiento:
 1. Texto1: gato juega en el jardín (clase: gato)
 2. Texto2: perro come hueso (clase: perro)
 3. Texto3: gato duerme en el sofá (clase: gato)
 4. Texto4: perro protege la casa (clase: perro)
 5. Texto5: gato y perro son amigos (clase: gato)
 6. Texto6: perro ladra fuerte (clase: perro)
 7. Texto7: gato salta alto (clase: gato)
- ▶ Clases: gato, perro
- ▶ Total de documentos: 7
- ▶ Documentos por clase: gato 4, perro 3

Paso 2: Construir el vocabulario

- ▶ Vocabulario (palabras únicas del train set):

gato, juega, en, el, jardín, perro, come, hueso, duerme, sofá,
y, son, amigos, protege, la, casa, ladra, fuerte, salta, alto

- ▶ Tamaño del vocabulario $|V| = 20$

Paso 3: Contar ocurrencias por clase

- ▶ Clase gato (total palabras = 16):

gato:4, juega:1, en:2, el:1, jardín:1, duerme:1, sofá:1,
y:1, son:1, amigos:1, salta:1, alto:1

- ▶ Clase perro (total palabras = 10):

perro:4, come:1, hueso:1, protege:1, la:1, casa:1, ladra:1, fuerte:1

Paso 4: Probabilidades a priori

$$P(\text{gato}) = \frac{4}{7} \approx 0.571, \quad P(\text{perro}) = \frac{3}{7} \approx 0.429$$

Paso 5: Probabilidades condicionales con Laplace smoothing

$$P(w|C) = \frac{\text{conteo de } w \text{ en } C + 1}{\text{total palabras en } C + |V|}$$

Ejemplos de Probabilidades Condicionales

- ▶ Palabra “gato”:

$$P(\text{gato}|\text{gato}) = \frac{4 + 1}{16 + 20} = 0.139,$$

$$P(\text{gato}|\text{perro}) = \frac{0 + 1}{10 + 20} = 0.033$$

- ▶ Palabra “perro”:

$$P(\text{perro}|\text{gato}) = \frac{0 + 1}{16 + 20} = 0.028,$$

$$P(\text{perro}|\text{perro}) = \frac{4 + 1}{10 + 20} = 0.167$$

- ▶ Palabra “duerme”:

$$P(\text{duerme}|\text{gato}) = \frac{1 + 1}{16 + 20} = 0.067,$$

$$P(\text{duerme}|\text{perro}) = \frac{0 + 1}{10 + 20} = 0.033$$

Paso 6: Clasificación de documentos de test

Documento 1: “gato persigue ratón”

- Probabilidad a priori:

$$P(\text{gato}) = 0.571, \quad P(\text{perro}) = 0.429$$

- Probabilidades condicionales (con Laplace smoothing):

$$P(\text{gato}|\text{gato}) = 0.139, \quad P(\text{persigue}|\text{gato}) = 0.028, \quad P(\text{ratón}|\text{gato}) = 0.028$$

$$P(\text{gato}|\text{perro}) = 0.033, \quad P(\text{persigue}|\text{perro}) = 0.033, \quad P(\text{ratón}|\text{perro}) = 0.033$$

- Multiplicando las probabilidades para obtener la probabilidad conjunta:

$$P(\text{doc1}|\text{gato}) = P(\text{gato}|\text{gato}) \cdot P(\text{persigue}|\text{gato}) \cdot P(\text{ratón}|\text{gato})$$

$$= 0.139 \cdot 0.028 \cdot 0.028 \approx 1.1 \cdot 10^{-4}$$

$$P(\text{doc1}|\text{perro}) = P(\text{gato}|\text{perro}) \cdot P(\text{persigue}|\text{perro}) \cdot P(\text{ratón}|\text{perro})$$

$$= 0.033 \cdot 0.033 \cdot 0.033 \approx 3.6 \cdot 10^{-5}$$

- ▶ Multiplicando por la a priori para obtener la probabilidad posterior:

$$P(\text{gato}|\text{doc1}) \propto 0.571 \cdot 1.1 \cdot 10^{-4} \approx 6.2 \cdot 10^{-5}$$

$$P(\text{perro}|\text{doc1}) \propto 0.429 \cdot 3.6 \cdot 10^{-5} \approx 1.5 \cdot 10^{-5}$$

- ▶ **Predicción final:** $\hat{y} = \text{gato}$ porque tiene mayor probabilidad posterior.

Resultado de Clasificación.

Resultados por muestra en el conjunto de test:

Muestra 1 (índice original 8):

Texto: gato persigue ratón

Clase real: gato

Clase predicha: gato

Probabilidades -> Prob(gato): 0.794, Prob(perro): 0.206

Muestra 2 (índice original 1):

Texto: perro corre en el parque

Clase real: perro

Clase predicha: gato

Probabilidades -> Prob(gato): 0.707, Prob(perro): 0.293

Muestra 3 (índice original 5):

Texto: perro juega con pelota

Clase real: perro

Clase predicha: perro

Probabilidades -> Prob(gato): 0.391, Prob(perro): 0.609

Ver código python

Bernoulli Naive Bayes

- ▶ Se usa para variables binarias (0/1), indicando presencia o ausencia:

$$P(\mathbf{x} \mid C_k) = \prod_{j=1}^d \theta_{kj}^{x_j} (1 - \theta_{kj})^{1-x_j}$$

- ▶ Donde:
 - ▶ $x_j \in \{0, 1\}$ indica ausencia/presencia de la característica j .
 - ▶ $\theta_{kj} = P(x_j = 1 \mid C_k)$ es la probabilidad de presencia de la característica j en la clase C_k .
- ▶ Predicción final:

$$\hat{y} = \arg \max_k P(C_k) \prod_{j=1}^d \theta_{kj}^{x_j} (1 - \theta_{kj})^{1-x_j}$$

Resumen Comparativo

Modelo	Tipo de dato	Probabilidad condicional
GaussianNB	Continuo	$P(x_j C_k) \sim \mathcal{N}(\mu_{kj}, \sigma_{kj}^2)$
MultinomialNB	Discreto (conteos)	$P(\mathbf{x} C_k) = \prod_j \theta_{kj}^{x_j}$
BernoulliNB	Binario	$P(\mathbf{x} C_k) = \prod_j \theta_{kj}^{x_j} (1 - \theta_{kj})^{1-x_j}$

Ventajas de Naive Bayes.

- ▶ Muy **rápido** de entrenar y predecir.
- ▶ Funciona bien con conjuntos de datos pequeños.
- ▶ Maneja bien datos discretos y continuos.
- ▶ Requiere **poca memoria**.
- ▶ Buen desempeño incluso cuando la independencia condicional es solo aproximada.

Desventajas de Naive Bayes.

- ▶ La suposición de independencia condicional rara vez se cumple en la práctica.
- ▶ Puede ser sensible a atributos con distribuciones no normales (en Gaussian NB).
- ▶ **No captura interacciones complejas entre características.**

Técnicas de **Regresión** Supervisada.

Regresión Lineal

- ▶ Modelo de aprendizaje supervisado utilizado para **predecir valores continuos**.
- ▶ Asume que existe una relación lineal entre las variables independientes y la variable dependiente:

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$

Ejemplo: predecir el precio de una vivienda según sus características.

Fundamento matemático.

Función de Decisión:

- ▶ La regresión lineal **predice el valor esperado de una variable continua y dado un vector de características $\mathbf{x}$:**

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$

- ▶ La predicción es una **combinación lineal de las variables de entrada.**
- ▶ El modelo asume que los errores siguen una distribución normal con media cero.

Función de Pérdida (Error Cuadrático Medio):

- Para ajustar los parámetros se minimiza la siguiente función de pérdida sobre N observaciones:

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{2N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

donde $\hat{y}_i = \mathbf{w}^\top \mathbf{x}_i + b$ es la predicción del modelo para la observación i .

Interpretación:

- **Penaliza los errores grandes** más severamente debido al cuadrado de las diferencias.

Optimización - Derivadas Parciales:

- Derivadas de la función de pérdida respecto a los parámetros:

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i) x_{ij}, \quad \frac{\partial \mathcal{L}}{\partial b} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)$$

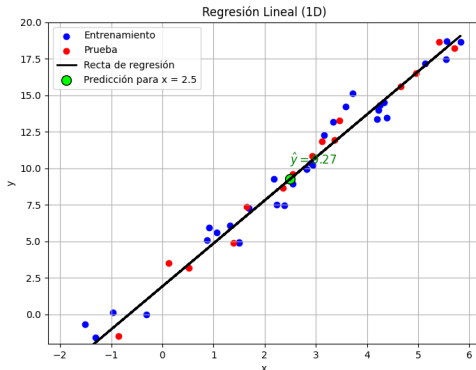
Actualización:

- Con gradiente descendente:

$$w_j \leftarrow w_j - \eta \frac{\partial \mathcal{L}}{\partial w_j}, \quad b \leftarrow b - \eta \frac{\partial \mathcal{L}}{\partial b}$$

donde η es la tasa de aprendizaje.

Ejemplo.



Dado el modelo $\hat{y} = 2.94x + 1.91$, para $x = 2.50 \Rightarrow \hat{y} \approx 9.27$.

Interpretación: el modelo estima un valor continuo de salida (ej. precio, temperatura, etc.) con base en variables predictoras.

Ver código python.

Máquinas de Vectores de Soporte para Regresión (SVR)

- ▶ SVR extiende las SVM al problema de **regresión**.
- ▶ El objetivo es encontrar una función que tenga como mucho una desviación ε de los valores reales para todos los puntos de entrenamiento.
- ▶ Se permite cierta tolerancia a errores mediante variables de holgura.

Ejemplo: predecir el precio de una casa dado su tamaño.

Función de Predicción:

- ▶ Dado un conjunto de entrenamiento $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$:

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b$$

- ▶ SVR busca minimizar la complejidad del modelo (el tamaño de $\|\mathbf{w}\|$) mientras mantiene los errores dentro de un margen ε :

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{sujeto a} \quad |y^{(i)} - \mathbf{w}^\top \mathbf{x}^{(i)} - b| \leq \varepsilon$$

Función de Costo Regularizada.

- ▶ Introduce variables de holgura ξ_i, ξ_i^* para puntos fuera del margen ε :

$$\min_{\mathbf{w}, b, \xi_i, \xi_i^*} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N (\xi_i + \xi_i^*)$$

sujeto a:

$$\begin{cases} y^{(i)} - (\mathbf{w}^\top \mathbf{x}^{(i)} + b) \leq \varepsilon + \xi_i \\ (\mathbf{w}^\top \mathbf{x}^{(i)} + b) - y^{(i)} \leq \varepsilon + \xi_i^* \\ \xi_i, \xi_i^* \geq 0 \end{cases}$$

- ▶ C : controla el equilibrio entre el margen plano y la penalización por error.

Dual de SVR.

- Se resuelve usando el problema dual, mediante **métodos de optimización cuadrática**, similar al caso de clasificación:

$$\begin{aligned} \max_{\alpha_i, \alpha_i^*} & \left(-\frac{1}{2} \sum_{i,j} (\alpha_i - \alpha_i^*)(\alpha_j - \alpha_j^*) K(\mathbf{x}_i, \mathbf{x}_j) \right. \\ & \left. + \varepsilon \sum_i (\alpha_i + \alpha_i^*) - \sum_i y_i (\alpha_i - \alpha_i^*) \right) \end{aligned}$$

sujeto a:

$$\sum_i (\alpha_i - \alpha_i^*) = 0, \quad 0 \leq \alpha_i, \alpha_i^* \leq C$$

- Las soluciones α_i, α_i^* determinan los vectores de soporte.

Predicción con Núcleos en SVR.

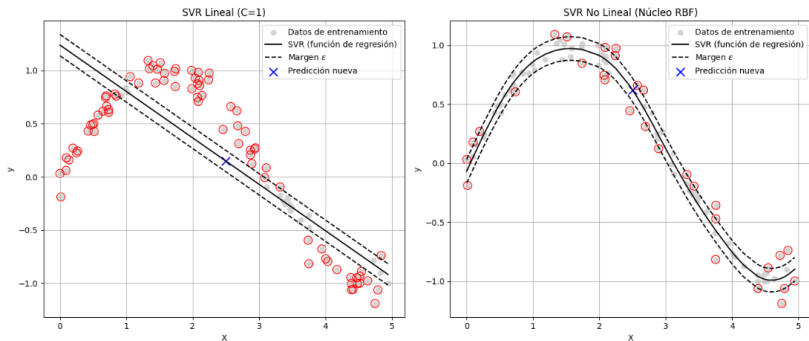
- ▶ La función de predicción en SVR con kernel es:

$$\hat{y} = \sum_{i=1}^N (\alpha_i - \alpha_i^*) K(\mathbf{x}_i, \mathbf{x}) + b$$

- ▶ El modelo depende únicamente de los **vectores de soporte**.
- ▶ Los kernels permiten modelar relaciones no lineales sin transformar explícitamente los datos:

$$K(\mathbf{x}, \mathbf{x}') = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle$$

Ejemplo Gráfico de SVR.



- ▶ Línea negra: función de regresión.
- ▶ Líneas punteadas: margen ϵ .

Ver código python.

Regresor KNN

El algoritmo KNN para regresión estima el valor de una muestra desconocida **basándose en el promedio de las etiquetas de sus k vecinos más cercanos** en el conjunto de entrenamiento.

- ▶ Método no paramétrico y basado en instancias.
- ▶ No se asume una forma funcional de la relación entrada-salida.

Predicción Vectorial.

- ▶ Dado un punto de prueba $\mathbf{x} \in \mathbb{R}^d$, y
- ▶ un conjunto de entrenamiento con salidas vectoriales $\mathbf{y}_i \in \mathbb{R}^m$ (por ejemplo, en regresión multisalida),
- ▶ la predicción se calcula como:

$$\hat{\mathbf{y}} = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{x})} \mathbf{y}_i$$

donde:

- ▶ $\hat{\mathbf{y}} \in \mathbb{R}^m$ es la predicción vectorial para $\mathbf{x}$.
- ▶ $\mathcal{N}_k(\mathbf{x})$ es el conjunto de índices de los k vecinos más cercanos a $\mathbf{x}$, según alguna métrica de distancia.
- ▶ $\mathbf{y}_i \in \mathbb{R}^m$ es el vector de salida del vecino i .

Parámetros Importantes.

- ▶ **Número de vecinos k :** controla la suavidad del modelo.
 - ▶ k pequeño: modelo muy flexible, pero sensible al ruido.
 - ▶ k grande: modelo más suave, pero puede perder detalles locales.
- ▶ **Métrica de distancia:**
 - ▶ Euclidiana: $\|\mathbf{x}_i - \mathbf{x}_j\|_2$
 - ▶ Manhattan: $\|\mathbf{x}_i - \mathbf{x}_j\|_1$
- ▶ **Ponderación:** vecinos pueden tener igual peso o ponderación inversa a la distancia.

Ventajas y Desventajas de KNN.

Ventajas:

- ▶ Sencillo de implementar e interpretar.
- ▶ No requiere entrenamiento explícito.
- ▶ Funciona bien con fronteras complejas si hay suficiente densidad de datos.

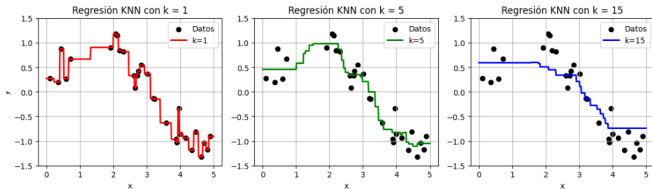
Desventajas.

- ▶ **Lento en predicción si hay muchos datos.**
- ▶ Sensible a la escala de las variables.
- ▶ Desempeño depende fuertemente de la elección de k .

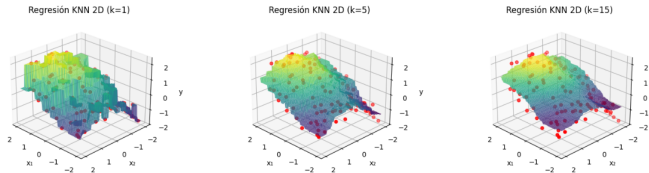
Impacto de k en la Suavidad

- ▶ $k = 1$: se ajusta a cada punto, puede sobreajustar el ruido.
- ▶ k **grande**: realiza una suavización global.

Efecto del valor de k en la Regresión KNN



Efecto del valor de k en la regresión KNN bidimensional



Ver código python.

Árboles de Decisión para Regresión

- ▶ Los árboles de decisión no solo se aplican a clasificación, también se utilizan en **regresión**.
- ▶ La idea es dividir el espacio de características en regiones y asignar un valor de predicción (promedio de los datos en la región).
- ▶ Cada nodo interno realiza una división basada en una variable de entrada y un umbral.

Ejemplo: predecir el precio de una casa según tamaño y número de habitaciones.

Construcción del Árbol de Regresión.

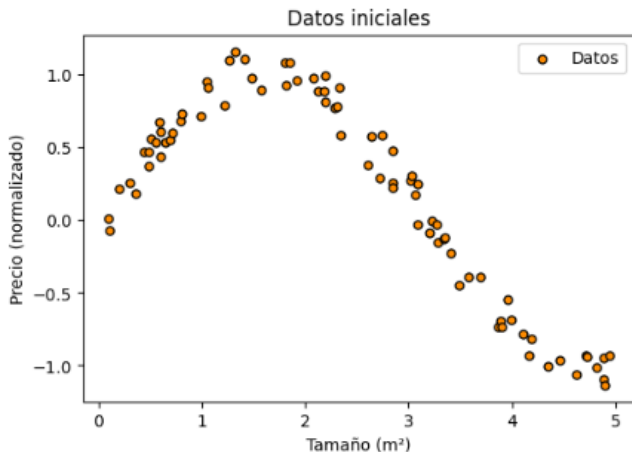
- ▶ Se parte de un conjunto de entrenamiento $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$.
- ▶ En cada nodo, se selecciona la variable x_j y el umbral s que minimicen el **error cuadrático medio** (ECM):

$$\min_{j,s} \left[\sum_{x_i \in R_1(j,s)} (y_i - \hat{y}_{R_1})^2 + \sum_{x_i \in R_2(j,s)} (y_i - \hat{y}_{R_2})^2 \right]$$

- ▶ Donde R_1, R_2 son las regiones creadas por la partición y $\hat{y}_{R_k}$ es el valor promedio de y en cada región.

Ejemplo:

- Sea un conjunto de datos (x_i, y_i) que representan el tamaño de una casa (x) y su precio (y).



Primer Cálculo de Partición

- Se selecciona la variable x_j y se busca el umbral s que minimicen el **error cuadrático medio** (ECM), lo que divide los datos en dos regiones:

$$R_1(s) = \{x \mid x \leq s\}, \quad R_2(s) = \{x \mid x > s\}$$

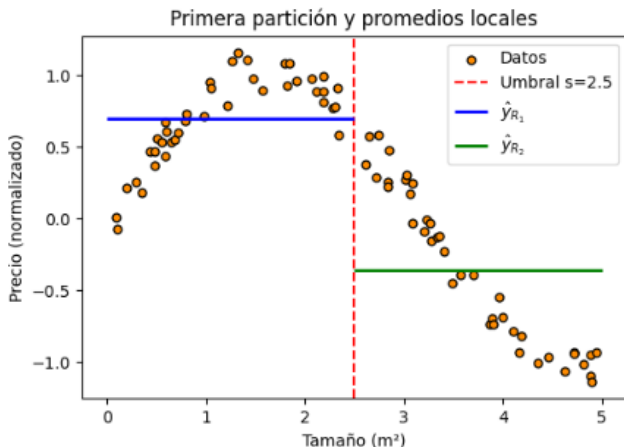
- Cada región R_k es aproximada por el promedio local $\hat{y}_{R_k}$:

x	y	Región
0.5	0.48	R_1
1.2	0.93	R_1
3.8	-0.63	R_2
4.5	-0.98	R_2

$$\hat{y}_{R_1} = \frac{0.48 + 0.93}{2} = 0.705, \quad \hat{y}_{R_2} = \frac{-0.63 + (-0.98)}{2} = -0.805$$

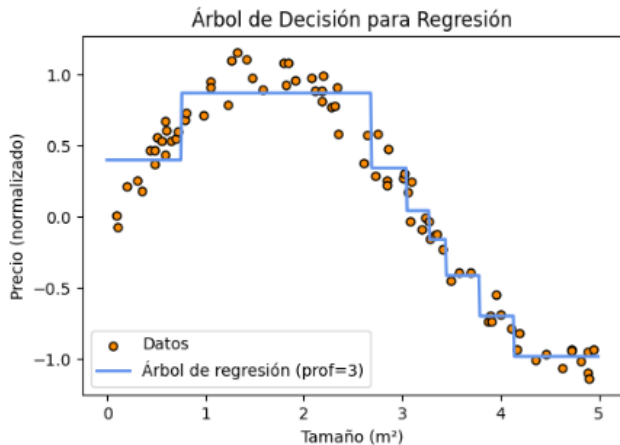
Visualización de la Partición

- La predicción tiene forma de escalón en el punto de corte s .



Árbol de Regresión con Mayor Profundidad

- ▶ Con más divisiones, el árbol aproxima mejor la relación entre x y y .
- ▶ La predicción final es una **función escalonada**.



Función de Predicción.

- ▶ El árbol de regresión divide el espacio en M regiones disjuntas $R_1, R_2, \dots, R_M$.
- ▶ La predicción final es:

$$\hat{f}(\mathbf{x}) = \sum_{m=1}^M \hat{c}_m \mathbf{1}_{\{\mathbf{x} \in R_m\}}$$

- ▶ Donde $\hat{c}_m = \frac{1}{|R_m|} \sum_{x_i \in R_m} y_i$, es el promedio de la variable respuesta en cada región.

Criterio de Partición.

- ▶ El criterio más utilizado es minimizar el **error cuadrático medio**:

$$ECM = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{f}(x_i))^2$$

- ▶ Cada partición busca reducir al máximo la variabilidad dentro de las regiones creadas.
- ▶ El árbol crece hasta alcanzar cierta profundidad o hasta que el número de datos en un nodo sea menor que un umbral.

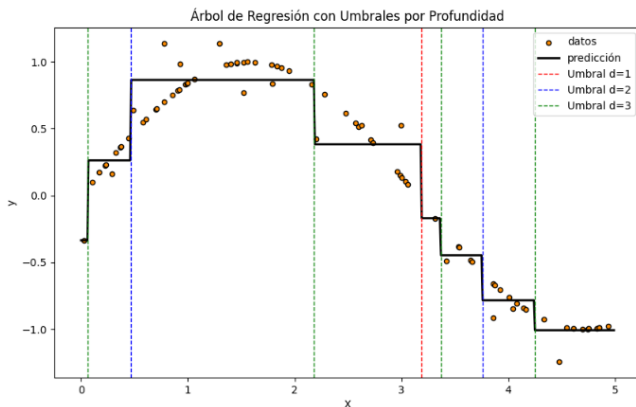
Regularización y Poda.

- ▶ Los árboles de regresión tienden a **sobreajustar** si se permiten crecer sin control.
- ▶ Se emplea un proceso de **poda** para reducir el tamaño del árbol:

$$C_{\alpha}(T) = \sum_{m=1}^{|T|} N_m Q_m(T) + \alpha |T|$$

- ▶ Donde $Q_m(T)$ es el error en el nodo m , $|T|$ es el número de hojas y α controla la complejidad.

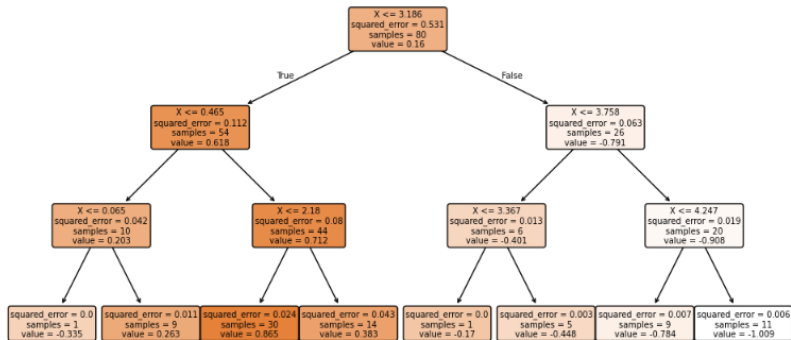
Ejemplo Gráfico de un Árbol de Regresión.



- ▶ Las divisiones horizontales representan umbrales en los atributos.
- ▶ Cada región se aproxima con el valor promedio de los datos que contiene.

Ejemplo Gráfico de un Árbol de Regresión.

Árbol de Decisión - Regresión (Profundidad 3)



Ver código python.

Random Forest para Regresión

- ▶ Es un **método de ensamble** que combina múltiples árboles de regresión.
- ▶ Cada árbol se entrena sobre un subconjunto de datos y variables (**bootstrap + submuestreo de características**).
- ▶ La predicción final es el **promedio** de las predicciones de todos los árboles.

Ejemplo: predecir el precio de una casa usando varios árboles de regresión y promediando sus resultados.

Construcción de un Random Forest.

- ▶ A partir de un conjunto de entrenamiento $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$:
 1. Se generan B subconjuntos de datos mediante **bootstrap**.
 2. Para cada subconjunto, se entrena un árbol de regresión.
 3. En cada división del árbol, se selecciona un subconjunto aleatorio de características.
- ▶ Esto introduce **diversidad** entre los árboles, reduciendo el sobreajuste.

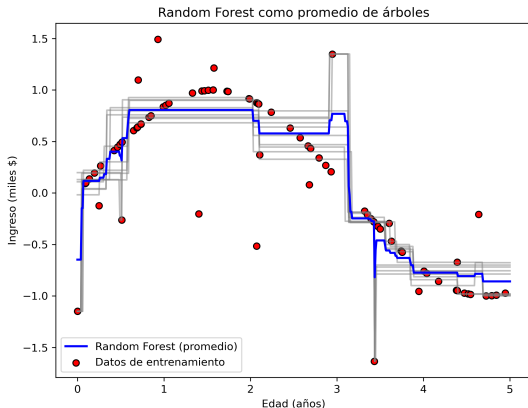
Predicción del Random Forest

- ▶ Dado un nuevo dato $\mathbf{x}$, cada árbol T_b produce una predicción $\hat{f}_b(\mathbf{x})$.
- ▶ La predicción final es el **promedio**:

$$\hat{f}_{RF}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(\mathbf{x})$$

- ▶ Esto reduce la varianza respecto a un único árbol.

Ejemplo Visual de Random Forest.



- ▶ Cada línea gris representa la predicción de un árbol.
- ▶ La línea azul es el **promedio**, más estable y preciso.

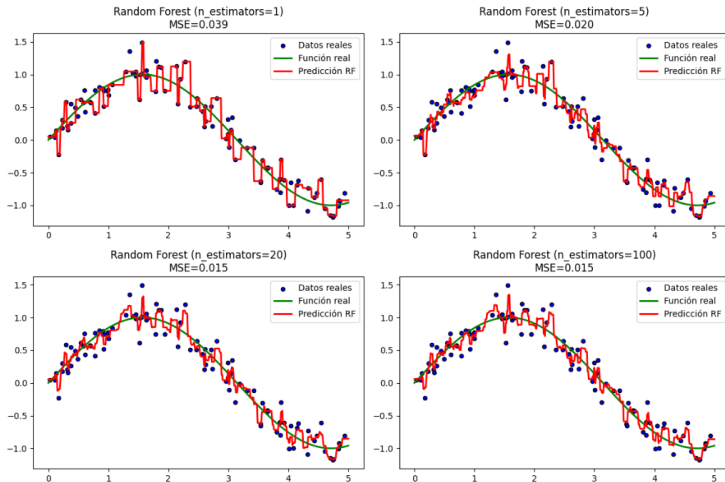
Criterio de Partición en cada Árbol.

- ▶ Cada árbol se construye minimizando el **error cuadrático medio** (ECM) en sus particiones:

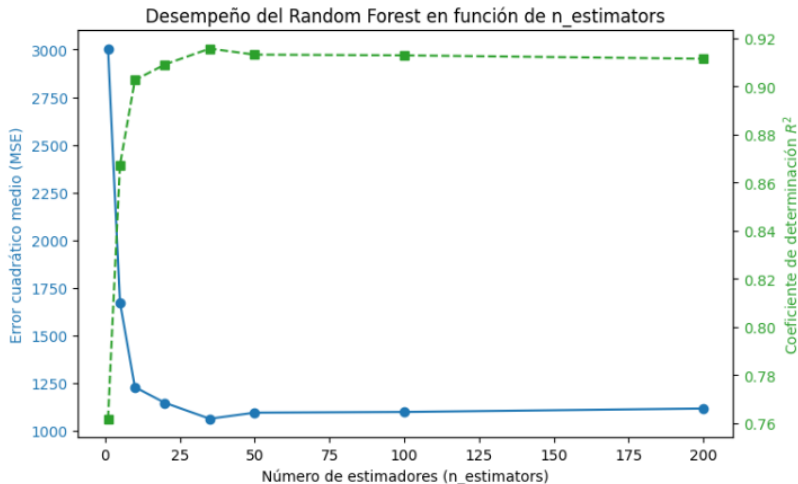
$$ECM = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{f}(x_i))^2$$

- ▶ Igual que en un árbol de regresión, pero con el beneficio de la diversidad al combinar múltiples árboles.

Número de estimadores (árboles) y desempeño.



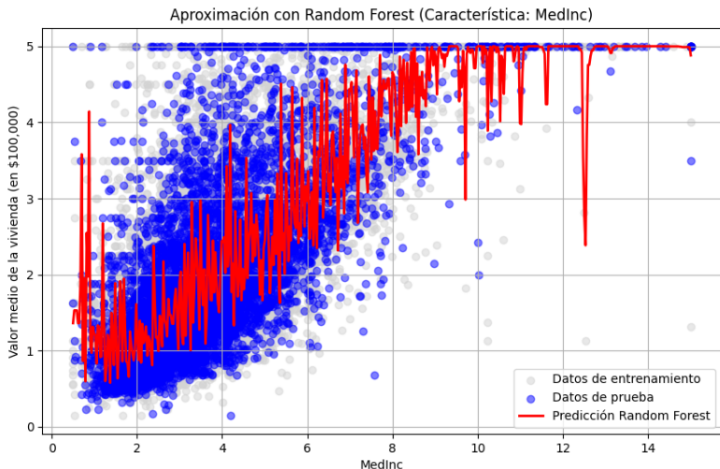
Número de estimadores (árboles) y desempeño.



Ventajas del Random Forest en Regresión

- ▶ **Robusto** frente al sobreajuste comparado con un solo árbol.
- ▶ Capacidad de manejar **alta dimensionalidad**.
- ▶ Maneja **relaciones no lineales** y **interacciones** entre variables.
- ▶ Proporciona una estimación de la **importancia de las variables**.

Ejemplo Gráfico de Predicción con Random Forest.



Ver código python.

Gradient Boosting para Regresión

- ▶ Es un **método de ensamble secuencial** que construye árboles de regresión de manera aditiva.
- ▶ Cada árbol corrige los **errores residuales** del modelo anterior.
- ▶ La predicción final es una combinación ponderada de árboles débiles (**boosting**).

Ejemplo: predecir el precio de una casa ajustando iterativamente los errores de modelos previos.

Construcción de un Gradient Boosting.

- ▶ Dado un conjunto de entrenamiento $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$:

1. Inicializar el modelo con una predicción base:

$$F_0(x) = \arg \min_c \sum_{i=1}^N L(y^{(i)}, c)$$

2. Para cada iteración $m = 1, \dots, M$:

- ▶ Calcular los **residuales** como el gradiente negativo de la función de pérdida: $r_i^{(m)} = - \left[\frac{\partial L(y^{(i)}, F(x^{(i)}))}{\partial F(x^{(i)})} \right]_{F=F_{m-1}}$
- ▶ Ajustar un árbol de regresión $h_m(x)$ a los residuales: $h_m(x) \approx r_i^{(m)}$
- ▶ Actualizar el modelo:

$$F_m(x) = F_{m-1}(x) + \eta h_m(x)$$

- ▶ Donde η es la **tasa de aprendizaje**.

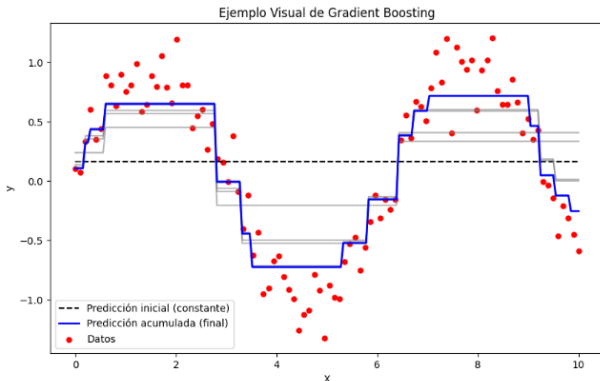
Predicción con Gradient Boosting

- ▶ La predicción final es la suma ponderada de árboles:

$$\hat{f}_{GB}(x) = F_M(x) = F_0(x) + \sum_{m=1}^M \eta h_m(x)$$

- ▶ Cada nuevo árbol **reduce el error** de la predicción previa.
- ▶ Esto permite mejorar tanto el **sesgo** como la **varianza**.

Ejemplo Visual de Gradient Boosting.



- Cada línea gris es un **árbol débil** que corrige errores del anterior.
- La línea azul es la predicción acumulada del modelo.

Función de Pérdida en Regresión.

- ▶ Generalmente se usa el **Error Cuadrático Medio (ECM)**:

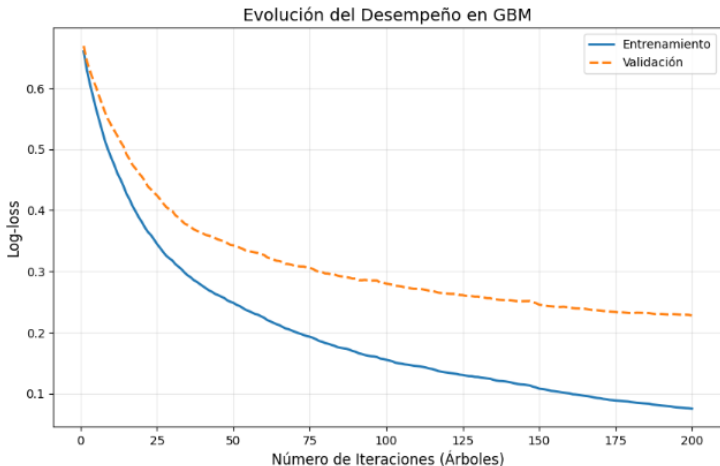
$$L(y, \hat{y}) = (y - \hat{y})^2$$

- ▶ Los residuales se calculan como:

$$r_m^{(i)} = -\frac{\partial L(y^{(i)}, F(x^{(i)}))}{\partial F(x^{(i)})} = y^{(i)} - F_{m-1}(x^{(i)})$$

- ▶ El árbol en la iteración m se ajusta para predecir $r_m^{(i)}$.

Número de Iteraciones y Desempeño.

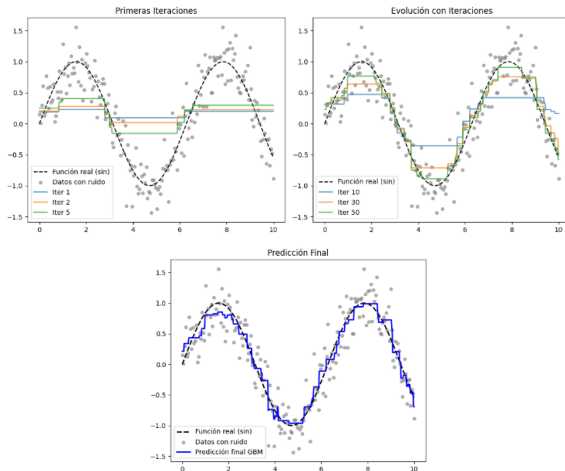


Más iteraciones → mayor ajuste, pero riesgo de **sobreajuste**.

Ventajas del Gradient Boosting en Regresión

- ▶ Alta **precisión predictiva**.
- ▶ Flexibilidad: admite diferentes funciones de pérdida.
- ▶ Captura relaciones **complejas y no lineales**.
- ▶ Permite controlar el sobreajuste mediante **learning rate** y número de iteraciones.

Ejemplo Gráfico.



Ver código python.

Gracias por su atención.

